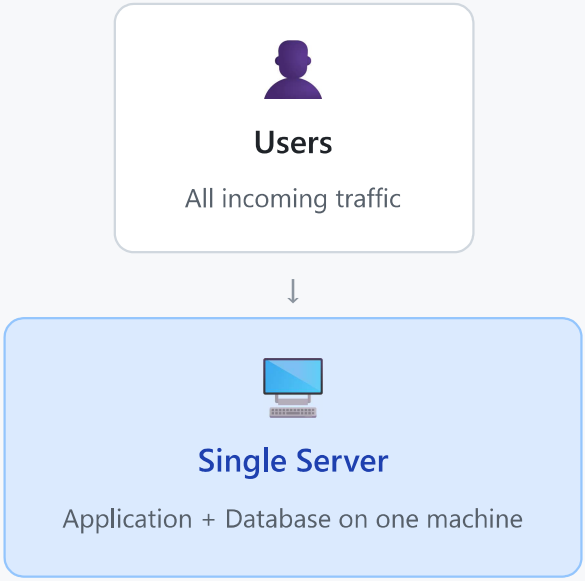
From Single Server to Distributed System

Step-by-step visual architecture evolution — vertical scaling, horizontal scaling, and distributed computing

1 Single Server STARTING POINT

One machine runs the application and the database. Simple to build and deploy — but a single point of failure.

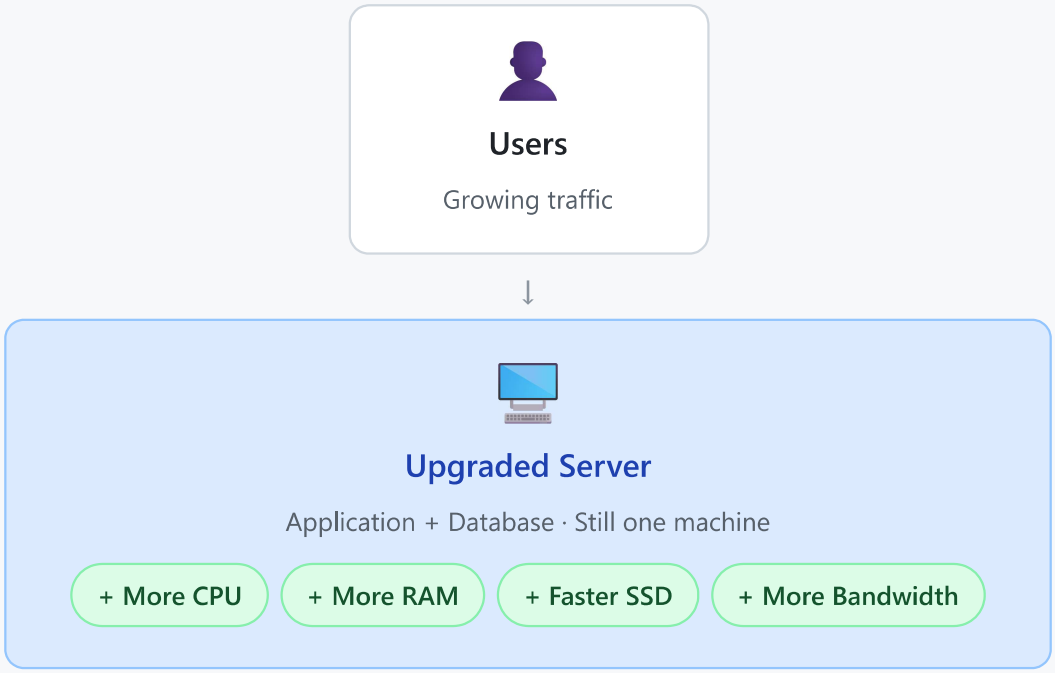


- ✓ Simple to develop and deploy
- ✓ Suitable for low traffic and prototypes
- ✗ Server becomes a bottleneck as users grow
- ✗ Single point of failure — everything goes down together

traffic grows → upgrade the machine

2 Vertical Scaling SCALE UP

Upgrade the same machine with more CPU cores, RAM, faster storage, or greater network capacity. No distribution needed — yet.

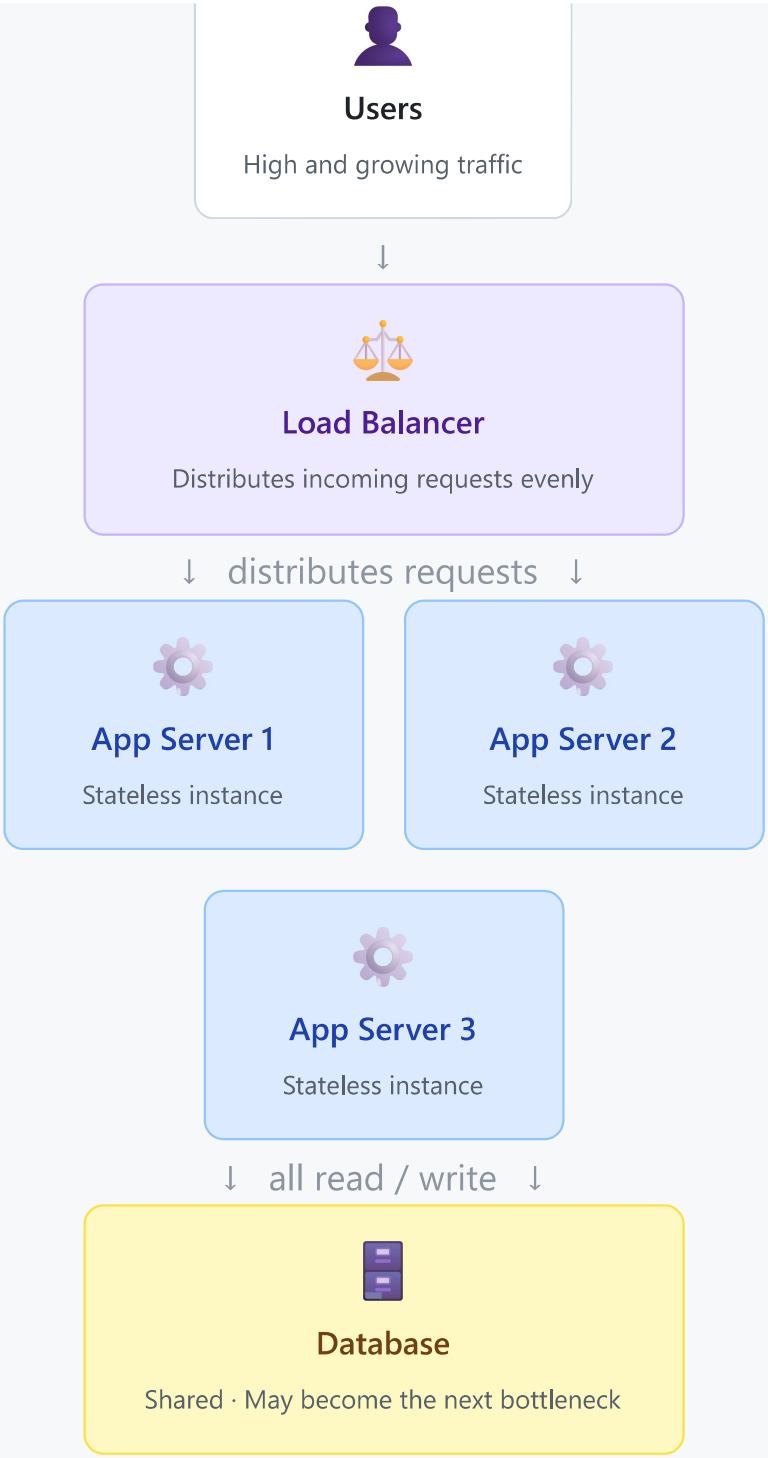


- ✓ No distributed complexity
- ✓ Simpler application code — no coordination needed
- ✓ Less operational overhead
- ✗ Hardware has a ceiling — eventually maxes out
- ✗ Downtime often required to upgrade hardware
- ✗ Still a single point of failure

traffic keeps growing → add more machines

3 Horizontal Scaling SCALE OUT

Add multiple application servers behind a load balancer. The database may still be a single shared node at this stage — it can become the next bottleneck.

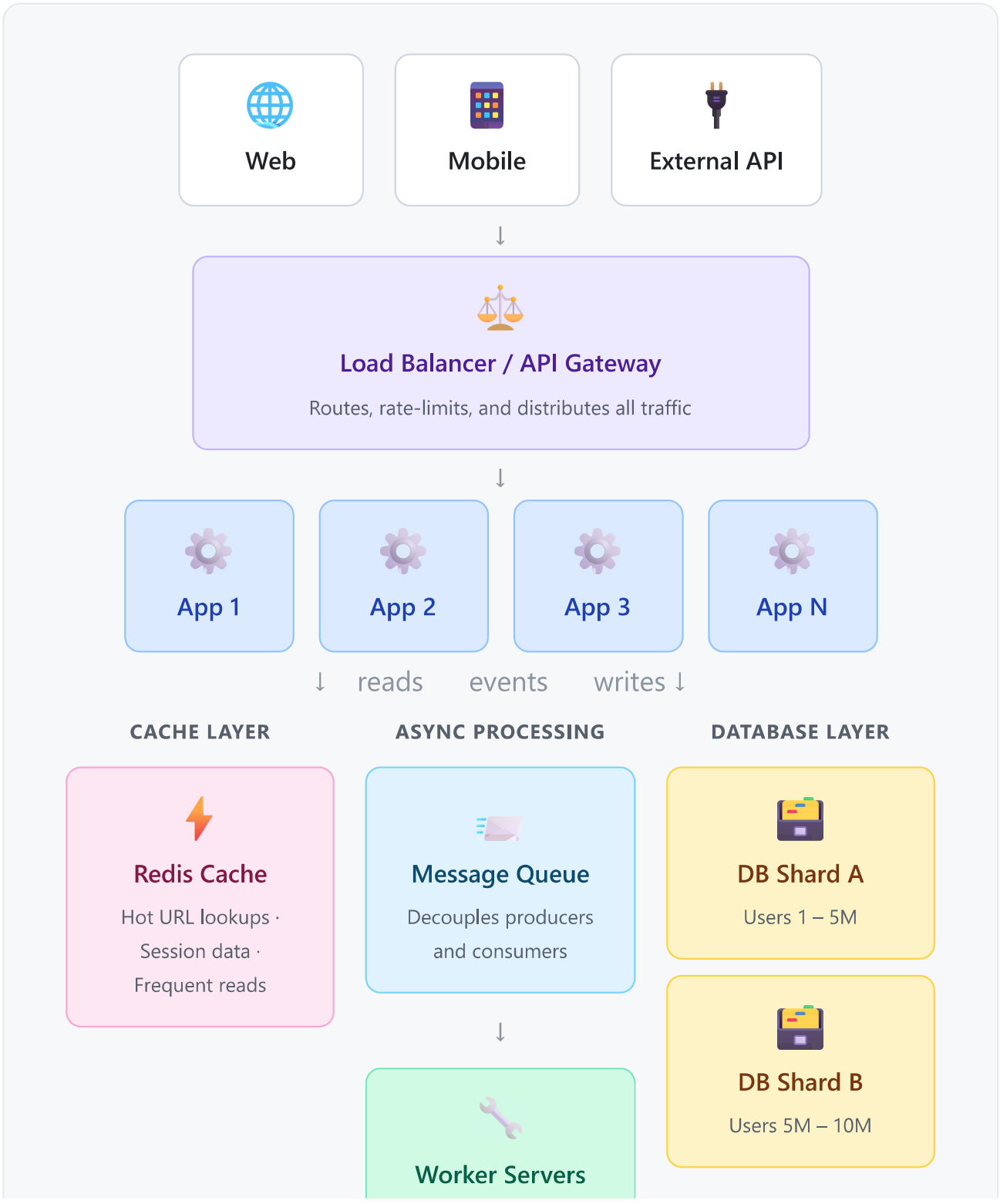


- ✓ Handles more concurrent requests
- ✓ Add or remove servers as demand changes
- ✓ Individual server failure does not take down the service
- ✗ Database can become the new bottleneck
- ✗ Stateless design required — sessions must be shared
- ✗ More operational complexity: monitoring, health checks

data volume grows → distribute storage too

4 Distributed Computing FULL SCALE

Both computation and storage are distributed. A cache layer absorbs read traffic. A message queue decouples async work. The database is partitioned and replicated.



Analytics · Emails · Background jobs



Read Replica

Scales read throughput

- ✓ Work processed in parallel across many machines
- ✓ Data partitioned (sharding) and replicated for scale
- ✓ Cache absorbs read traffic; queue decouples async work
- ✗ Network failures and partial failures must be managed
- ✗ Distributed coordination adds significant overhead
- ✗ Requires observability, tracing, and failure recovery

Quick Comparison

Factor	Single Server	Vertical Scaling	Horizontal Scaling	Distributed
Complexity	Very Low	Low	Medium	High
Capacity ceiling	Hardware	Hardware	App ✓, DB ✗	Can scale further
Availability	Single PoF	Single PoF	Redundant app layer	Highly available
Data management	Simple	Simple	Shared DB	Sharding + Replication
Maintenance	Easy	Hardware downtime	Rolling upgrades possible	Requires automation

Factor	Single Server	Vertical Scaling	Horizontal Scaling	Distributed
Best suited for	Prototype / MVP	Small–medium apps	Growing traffic	Large-scale production

---

NODE LEGEND

Application Server · Database · Load Balancer · Cache (Redis)

Message Queue · Worker Server · DB Shard / Replica